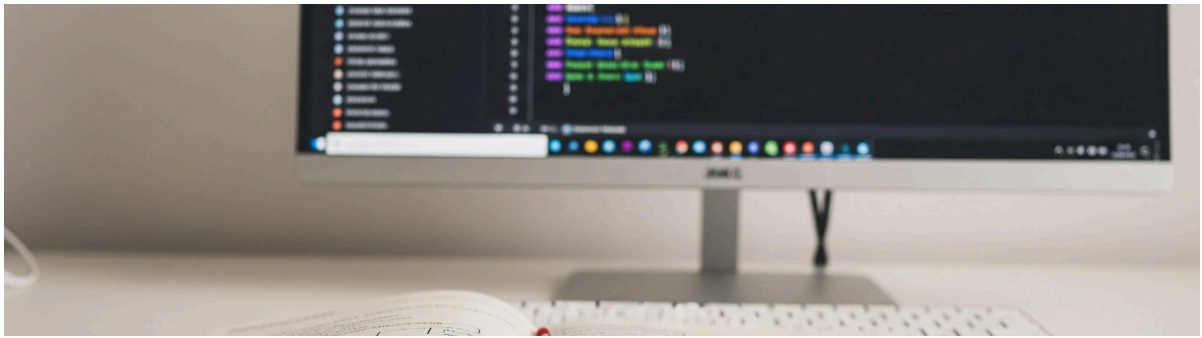




개발 일지(UART LOOPBACK)

2026.04.23 - UART 비동기 통신 아키텍처 분석 및 데이터 동기화 메커니즘 학습

작업 환경 : Notion

1. 작업 내용 (What I Did)

구분	내용
작업 내용 (What I Did)	UART 프로토콜의 내부 로직 분석 및 하드웨어 인터페이스 설계 원리 습득
상세 내역	- UART 프레임 구조 분석 : Start bit, 8-bit Data, Stop bit로 이어지는 비동기 데이터 프레임이 하드웨어 레벨에서 어떻게 생성되고 검출되는지 로직 확인 - Baud Rate 타이밍 제어 : 클럭을 공유하지 않는 두 장치간의 동기화 방식(Sampling) 및 오차 극복 원리 파악 - Baud Rate Generator 설계 : 시스템 클럭을 분주하여 원하는 통신 속도(9600 bps)를 생성하는 분주기 로직 검토

2. 핵심 기술 학습 내용

구분	내용
비동기 통신의 핵심 : 샘플링	클럭 신호가 별도로 없는 UART의 특성상, 수신 측(RX)에서 Start Bit를 감지한 후 Data Bit의 중앙 지점에서 샘플링하여 데이터의 신뢰성을 확보하는 것이 가장 중요하다는 점을 학습하였습니다.
16x 오버샘플링	단순히 1배수 속도로 읽는 것이 아니라, 16배수 클럭으로 샘플링하여 Start Bit의 하강 에지를 정밀하게 감지하고 노이즈에 강인한 수신 환경을 구축하였습니다.
프레임 구조	• Idle State : 통신이 없을 때는 High(1) 상태 유지 • Start Bit : Low(0) 신호로 데이터 전송 시작을 알립니다. • Data Bits : 8비트 데이터를 LSB부터 전송 • Stop Bit : High(1) 신호로 한 프레임의 종료를 알립니다.

3. 회고 및 개선 사항

구분	내용
Keep	타이밍 기반 설계의 중요성 : 통신 속도에 맞춰 정확한 시점에 데이터를 읽고 쓰는 타이밍 로직이 전체 시스템의 안정성을 결정한다는 점을 인지하고 설계에 반영한 점입니다.
Try	[샘플링 타이밍 정밀 검증] 16배속 오버샘플링 로직이 실제 시뮬레이션 파형에서 Start Bit 이후 정확히 8번째 틱(중앙 지점)에서 데이터를 저장하는지 타이밍 마진을 체크해볼 예정입니다.

4. 이슈 및 해결 과정

구분	내용
Trouble	[송수신 장치 간 클럭 오차 발생 가능성] 비동기 통신 특성상 송신 측과 수신 측의 클럭 속도가 미세하게 다를 경우, 데이터가 뒤로 갈수록 비트 밀림 현상이 발생하여 데이터가 깨지는 이슈가 우려됩니다.
해결 과정	Baud Rate 정밀 설정 및 샘플링 시점 최적화

구분

내용

Baud Tick Generator의 카운트 로직을 검토하여 오차를 최소화하였습니다. 특히, 16배속 오버샘플링을 통해 비트의 정중앙(8번째 틱)에서 데이터를 수집하도록 설계하여, 하드웨어적 클럭 오차나 노이즈에 대한 충분한 타이밍 마진을 확보하고 데이터 무결성을 유지합니다.